

Informe de Seguridad Estática SAST

OWASP Top 10 · Rutas /src/app/api/*

Fecha de emisión: **2026-08-12**

Report ID: **sast-api-20260812**

Engine: SAST Manual (Senior Pentest Review)

Project: **cfdi_taskmanager_demo** · Stack: Next.js 15 / Prisma / TypeScript



Resumen Ejecutivo de Riesgos



Hallazgos por categoría OWASP Top 10 2021

- **A01 Broken Access Control:** 6 (API-01, 02, 03, 04, 06, 05 parcial)
- **A02 Cryptographic Failures:** 2 (API-08, API-12)
- **A03 Injection:** 2 (API-09 mass+xml bomb, API-01 sin schema)
- **A05 Security Misconfiguration:** 3 (API-05, API-10, API-11)
- **A07 Identification & Auth Failures:** 2 (API-01 sin auth, API-02 path traversal)
- **A08 Software Integrity:** 1 (API-09 sin validación de integridad de XML)
- **A09 Logging Failures:** 2 (API-07 details leak, API-12 log stack)



Tabla de Contenidos · Hallazgos

ID	Riesgo	Título	Ubicación
API-01	Crítico	Ruta /api/import SIN AUTENTICACIÓN (escritura masiva de CFDI anónima)	src/app/api/import/route.ts
API-02	Crítico	/invoices/:id/pdf sin autenticación + Path Traversal (lectura de archivos arbitrarios)	src/app/api/invoices/[id]/pdf/route.ts
API-03	Crítico	/api/dev/sat_invoices SIN AUTENTICACIÓN NI LÍMITES (todas las facturas expuestas)	src/app/api/dev/sat_invoices/route.ts
API-04	Crítico	/mass-downloads/download-zip descarga paquetes de CUALQUIER RFC sin verificación de Tenant	src/app/api/mass-downloads/download-zip/route.ts
API-05	Crítico	/api/dev/seed ejecutable en PRODUCCIÓN (crea Orgs + M2M clients arbitrarios)	src/app/api/dev/seed/route.ts
API-06	Alto	/companies/:id/logo permite subir logo a CUALQUIER empresa de otra org	src/app/api/companies/[id]/logo/route.ts
API-07	Alto	Info Leak en respuestas 500: error.message en campo "details" expone stack/Prisma/SAT	src/app/api/companies/tenant/route.ts + /mass-downloads/download-zip/route.ts
API-08	Alto	/mass-downloads/credentials FIEL Upload sin validación de contenido (MIME Spoofing + DoS)	src/app/api/mass-downloads/credentials/route.ts
API-09	Alto	/api/import sin validación de tamaño ni schema (XML Bomb / Billion Laughs)	src/app/api/import/route.ts
API-10	Medio	Falta Rate Limit en endpoints de administración de queues y workers	src/app/api/admin/sat-69b/sync/route.ts, src/app/api/tenant/update-progress/route.ts

ID	Riesgo	Título	Ubicación
API-11	Medio	Endpoints /api/dev/* sin guardia de entorno global (apply a todo el segmento)	src/app/api/dev/ (toda la carpeta)
API-12	Medio	/api/import console.error stack FULL + errorMessage al cliente XML leak	src/app/api/import/route.ts

API-01

Ruta /api/import SIN AUTENTICACIÓN (escritura masiva de CFDI anónima)

Crítico OWASP A01:2021 · Broken Access Control

 src/app/api/import/route.ts L16-L87 (todo el handler POST) Archivo y Líneas exactas

src/app/api/import/route.ts @ L16-L87 (todo el handler POST)

 Nivel de Riesgo

Crítico

 Descripción del Riesgo

El endpoint POST /api/import NO invoca auth() en ningún punto. Permite que cualquier cliente sin sesión envíe un array de CFDI en XML y los persista directamente en la base de datos vía createInvoiceFromXml. No hay rate limit, no hay verificación de tenant, no hay validación de firma XML (válido o no). Equivale a una puerta trasera de data injection sobre el modelo core de facturas, capaz de sobrescribir UUIDs legítimos o inundar el sistema con millones de registros.

 Ejemplo de Exploit (PoC)

```
1. Sin iniciar sesión:
POST /api/import HTTP/1.1
Host: app.example.com
Content-Type: application/json
[
  {
    "xmlContent": "<?xml version='1.0'?><cfdi:Comprobante ... >\n<cfdi:Complemento>
<tfd:TimbreFiscalDigital UUID=\"00000000-0000-0000-0000-000000000001\" ... />
</cfdi:Complemento></cfdi:Comprobante>",
    "source_file": "factura_falsa.xml"
  },
  // ... 1 millón de registros usando Batching ...
]
```

Respuesta: success:true, results: [{uuid:"...", status:"created"}].

Impacto adicional: Denegación de servicio (almacenamiento ilimitado / procesamiento de parseo XML 100% CPU / RAM).

 Código Corregido

```
export async function POST(request: NextRequest) {
  try {
    const session = await auth()
    if (!session?.user?.id) {
      return NextResponse.json({ error: 'No autorizado' }, { status: 401 })
    }

    // [SAST-FIX API-01] Rate limit por usuario: máx. 10 lotes por hora
    rateLimitByUserId({
      userId: session.user.id,
      key: 'cfdi-import-batch',
      limit: 10,
      windowMs: 60 * 60 * 1000
    })

    // [SAST-FIX] Resolver tenant y limitar lote
    const member = await prisma.member.findFirst({
      where: { userId: session.user.id, status: 'APPROVED' },
      select: { organizationId: true, role: true, id: true }
    })
    if (!member) return NextResponse.json({ error: 'Sin membresía' }, { status: 403 })

    const body = importBatchSchema.parse(await request.json())
    if (body.length > 500) {
      return NextResponse.json(
        { error: 'Lote excede el tamaño máximo permitido (500 CFDI)' },
        { status: 413 }
      )
    }
  }
}
```

API-02

/invoices/:id/pdf sin autenticación + Path Traversal (lectura de archivos arbitrarios)

Crítico OWASP A01:2021 / A07:2021 · Broken Authentication + Path Traversal

src/app/api/invoices/[id]/pdf/route.ts L12-L45 (handler GET completo)

Archivo y Líneas exactas

src/app/api/invoices/[id]/pdf/route.ts @ L12-L45 (handler GET completo)

⚡ Nivel de Riesgo

Crítico

🔍 Descripción del Riesgo

La ruta GET /api/invoices/:id/pdf tiene DOS problemas independientes de severidad máxima: (1) NO llama auth() - cualquier usuario anónimo puede descargar el PDF de CUALQUIER factura del sistema conociendo su id (UUID enumeración reducida pero factible por fuerza bruta si hay leaks). (2) El query param ?file= concatena con path.join sin sanitizar SECUENCIAS RELATIVAS: la guardia L24-L26 SOLO detecta paths absolutos (C:\ o /), pero ../../windows/win.ini o ../../.env.local pasan sin problemas. Resultado: LFI (Local File Inclusion) → robo de código fuente, secrets, ENV vars, claves de cifrado, etc.

🌟 Ejemplo de Exploit (PoC)

Escenario 1 - Path Traversal ENV:

```
GET /api/invoices/any-uuid-random/pdf?file=../../../../../.env.local HTTP/1.1
```

Host: app

Respuesta: binario del archivo (renderiza si es .env como raw text, puppeteer intenta parsear como XML pero readFile L27 expone el error con contenido si el throw se filtra - incluso el 500 devuelve el buffer en escenarios de NextResponse mal estructurado).

Escenario 2 - Leak de XML de CFDI vecino:

```
GET /api/invoices/uuid-de-OTRA-empresa/pdf
```

Devuelve el PDF completo con datos fiscales de terceros sin verificar tenant ni permisos VIEWER del RFC.

✅ Código Corregido

```
export async function GET(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await auth()
```

```
if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status: 401 })

const { id } = await params

// [SAST-FIX API-02] ELIMINAR por completo el parámetro ?file= en PRODUCCIÓN.
// Si es absolutamente necesario para dev:
const fileParam = request.nextUrl.searchParams.get('file')
if (fileParam) {
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Parametro no permitido' }, { status: 400 })
  }
  // Safe join: resolver y verificar que esté dentro de java-client/xml-data
  const safeBaseDir = path.resolve(process.cwd(), 'java-client', 'xml-data')
  const candidate = path.resolve(safeBaseDir, fileParam)
  if (!candidate.startsWith(safeBaseDir + path.sep) && candidate !== safeBaseDir) {
    return NextResponse.json({ error: 'Path inválido' }, { status: 400 })
  }
  // Regex allowlist extension
  if (!/\.xml$/i.test(candidate)) {
    return NextResponse.json({ error: 'Solo archivos XML permitidos' }, { status: 400 })
  }
}
```


API-03

/api/dev/sat_invoices SIN AUTENTICACIÓN NI LÍMITES (todas las facturas expuestas)

Crítico

OWASP A01:2021 · Broken Access Control

 src/app/api/dev/sat_invoices/route.ts L4-L30 (handler GET completo)

Archivo y Líneas exactas

src/app/api/dev/sat_invoices/route.ts @ L4-L30 (handler GET completo)

Nivel de Riesgo

Crítico

Descripción del Riesgo

El endpoint GET /api/dev/sat_invoices no valida sesión, ni permisos, ni tenant. Expone directamente todos los registros satInvoice con issuerRfc, receiverRfc, nombre emisor/receptor, subtotal, total, moneda, UUID. El único filtro es ?rfc= que reduce el set a un RFC concreto; ?limit= acepta hasta 100. Cualquiera desde Internet podría scrapear toda la base de facturas vía paginación rfc por rfc (2^24 RFCs posibles, pero con los prefijos conocidos de CFDI registrados SAT se completaría en horas). Fuga masiva de PII fiscal.

Ejemplo de Exploit (PoC)

```
GET /api/dev/sat_invoices?limit=100 HTTP/1.1
```

```
Host: app
```

```
Response: { count:100, invoices: [ { uuid:"...", issuerRfc:"XXX010101XXX",
issuerName:"EMPRESA SA DE CV", receiverRfc:"YYY010101YYY", total:15450.22, currency:"MXN",
issuanceDate:"..." } ...] }
```

Ataque repetitivo: iterar ?limit=100&offset=X vía created_at cursor (no hay offset param pero iterar issuanceDate: lte cursor).

Código Corregido

```
export async function GET(request: NextRequest) {
  // [SAST-FIX API-03] Bloquear en producción rotundamente
  if (process.env.NODE_ENV === 'production') {
    return NextResponse.json({ error: 'Endpoint de desarrollo deshabilitado' }, { status:
404 })
  }

  try {
    const session = await auth()
```

```
if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status: 401 })

// Solo SUPER_ADMIN puede usarlo, ni siquiera ADMIN de tenant
const me = await prisma.user.findUnique({ where: { id: session.user.id }, select: { systemRole: true } })
if (me?.systemRole !== 'SUPER_ADMIN') {
  return NextResponse.json({ error: 'Permiso denegado' }, { status: 403 })
}

const { searchParams } = new URL(request.url)
const limit = Math.min(Number(searchParams.get('limit') || 10), 50)
const rfc = searchParams.get('rfc') || undefined
const where = rfc ? { OR: [{ issuerRfc: rfc }, { receiverRfc: rfc }] } : undefined
const rows = await prisma.satInvoice.findMany({
  where, orderBy: { issuanceDate: 'desc' }, take: limit,
  select: { id: true, uuid: true, cfdiType: true, issuerRfc: true, issuerName: true, receiverRfc: true, receiverName: true, subtotal: true, total: true, issuanceDate: true,
```

API-04

/mass-downloads/download-zip descarga paquetes de CUALQUIER RFC sin verificación de Tenant

Crítico

OWASP A01:2021 · Broken Access Control (IDOR)

 src/app/api/mass-downloads/download-zip/route.ts L12-L33 (GET handler)

Archivo y Líneas exactas

src/app/api/mass-downloads/download-zip/route.ts @ L12-L33 (GET handler)

Nivel de Riesgo

Crítico

Descripción del Riesgo

El endpoint acepta `?rfc=RFC_EMPRESA&idPaquete=PKT` y pasa los parámetros DIRECTAMENTE al servicio SAT `downloadMassPackages L21`. ANTES, solo chequea que el request tenga `session.user.id (L8)`, PERO NUNCA verifica que ese RFC (sujeto de la descarga) pertenezca a un `FiscalEntity` de la `organizationId` del usuario autenticado. Resultado: un usuario legítimo de Org-A puede descargar todos los paquetes masivos del SAT de Org-B (eavesdropping de XMLs de terceros con los credenciales FIEL del SAT del tenant del atacante - el WS acepta si los credenciales son válidos para el RFC solicitado).

Ejemplo de Exploit (PoC)

Usuario Autenticado (RFC VÁLIDO, FIEL configurada):

```
GET /api/mass-downloads/download-zip?rfc=RFC_EMPRESA_AJENA&idPaquete=xxx-xxx-xxx HTTP/1.1
Authorization: Bearer <su_token>
```

El sistema usa SU FIEL para autenticar al SAT y solicita el paquete de OTRA EMPRESA. Si el SAT lo entrega (falla de diseño del SAT) → el atacante obtiene el ZIP con los XMLs de terceros. Incluso si el SAT rechaza, el endpoint permite enumerar qué RFCs han solicitado descargas (timing attack y mensajes de error SAT diferenciados).

Código Corregido

```
export async function GET(request: NextRequest) {
  try {
    const session = await auth()
    if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status: 401 })

    rateLimitByUserId({ userId: session.user.id, key: 'sat-mass-download', limit: 30,
```

```
windowMs: 15 * 60 * 1000 })

const { searchParams } = new URL(request.url)
const rfc = searchParams.get('rfc')
const idPaquete = searchParams.get('idPaquete')
if (!rfc || !idPaquete) return NextResponse.json({ error: 'Faltan parámetros' }, {
status: 400 })

// [SAST-FIX API-04] Scoping estricto RFC ↔ Organization
const member = await prisma.member.findFirst({
  where: { userId: session.user.id, status: 'APPROVED' },
  select: { organizationId: true, id: true }
})
if (!member) return NextResponse.json({ error: 'Sin membresía' }, { status: 403 })

const fiscalAccess = await prisma.fiscalEntity.findFirst({
  where: { organizationId: member.organizationId, rfc, isActive: true },
  select: { id: true, rfc: true }
})
```

API-05

/api/dev/seed ejecutable en PRODUCCIÓN (crea Orgs + M2M clients arbitrarios)

Crítico

OWASP A05:2021 / A01:2021 · Security Misconfiguration + Elevation of Privilege

 src/app/api/dev/seed/route.ts L11-L83 (handler POST)

Archivo y Líneas exactas

src/app/api/dev/seed/route.ts @ L11-L83 (handler POST)

Nivel de Riesgo

Crítico

Descripción del Riesgo

El endpoint dev/seed se protege únicamente con session.user.id (L13-16). NO valida: (a) que NODE_ENV === "development" para bloquear ejecución en producción, (b) que el solicitante sea SUPER_ADMIN. Cualquier usuario registrado (free trial / onboarding) puede enviar un POST y: 1) crear una Organization nueva con ownerId = session.user.id; 2) generar un Machine client M2M con clientSecret en texto plano en la response; 3) crear una company APPROVED hardcodeda con RFC SCI041122EI6; 4) agregar ADMIN companyAccess y FiscalEntity. Resultado: elevación de privilegios sin restricciones (cada usuario se convierte en Owner de N organizaciones nuevas cada minuto = spam de tenants / agotamiento de cuotas / uso fraudulento de workers BullMQ).

Ejemplo de Exploit (PoC)

```
1. Iniciar sesión con cuenta free trial: user@attacker.com
2. POST /api/dev/seed (repetir 100 veces):
→ 100 organizaciones owner → 100 M2M clientSecret en claro → 100 companies APPROVED → 100
FIEL slots para abusar del SAT masivo.
```

Código Corregido

```
export async function POST() {
  // [SAST-FIX API-05] Guardia irrompible de entorno
  if (process.env.NODE_ENV !== 'development') {
    return NextResponse.json({ error: 'Endpoint no disponible' }, { status: 404 })
  }
  try {
    const session = await auth()
    if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status: 401 })

    // [SAST-FIX API-05] Solo SUPER_ADMIN puede ejecutar seed dev
```

```
const me = await prisma.user.findUnique({ where: { id: session.user.id }, select: {
systemRole: true } })
if (me?.systemRole !== 'SUPER_ADMIN') {
  return NextResponse.json({ error: 'Sin privilegios para seed' }, { status: 403 })
}

// Idempotency: evitar que se ejecuten 2 seeds para el mismo owner en menos de 60 s
// ... resto del código, CON user.id SUPER_ADMIN ...
return NextResponse.json({ success: true, org: org.id })
} catch (error) {
  console.error('[dev-seed]', error)
  return NextResponse.json({ error: 'Error interno' }, { status: 500 })
}
}
```

API-06

/companies/:id/logo permite subir logo a CUALQUIER empresa de otra org

Alto OWASP A01:2021 · Broken Access Control (IDOR)

 src/app/api/companies/[id]/logo/route.ts L37-L46 (validación de ownership) Archivo y Líneas exactas

src/app/api/companies/[id]/logo/route.ts @ L37-L46 (validación de ownership)

 Nivel de Riesgo

Alto

 Descripción del Riesgo

El endpoint POST valida permisos globales `Permission.COMPANY_UPDATE` (L39), pero nunca comprueba si la company con `id = params.id` pertenece a la organización del usuario. Basta con tener el permiso `COMPANY_UPDATE` (global) en el profile para poder: 1) sobrescribir el logo de una empresa de Tenant-B desde Tenant-A; 2) potencialmente abusar `allowedTypes` (aunque mime está chequeado) para denegación de servicio repitiendo uploads de 5MB. El id de Company es UUID v4 difícil de adivinar, pero se leakea por múltiples respuestas de API (`/api/companies/search` JSON list, `user/company-access`).

 Ejemplo de Exploit (PoC)

```
user@tenant-a.com tiene Permission.COMPANY_UPDATE en Tenant-A.
1. GET /api/user/company-access → obtiene lista companies de Tenant-A.
2. Obtiene company-UUID de Tenant-B desde otra respuesta que hace leak accidental de UUIDs (ej: public search).
3. POST /api/companies/<uuid-TENANT-B>/logo multipart/form-data → logo sobrescrito.
Nombre de archivo: phishing-corporativo.png.
Impacto: defacement de empresa de terceros / phishing en dashboards de partners.
```

 Código Corregido

```
export async function POST(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const session = await auth()
    if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status: 401 })
    const { id } = await params
```

```
// [SAST-FIX API-06] Validar tenant de la empresa
const member = await prisma.member.findFirst({
  where: { userId: session.user.id, status: 'APPROVED' },
  select: { organizationId: true, role: true }
})
if (!member) return NextResponse.json({ error: 'Sin membresía' }, { status: 403 })

const company = await prisma.company.findUnique({
  where: { id },
  select: {
    id: true,
    companyAccesses: {
      where: { organizationId: member.organizationId },
      take: 1, select: { id: true, role: true }
    }
  }
})
```


API-07

Info Leak en respuestas 500: error.message en campo "details" expone stack/Prisma/SAT

Alto OWASP A09:2021 · Security Logging & Monitoring Failures + Sensitive Data Exposure

src/app/api/companies/tenant/route.ts + /mass-downloads/download-zip/route.ts tenant L69-L74, download-zip L35-L40

Archivo y Líneas exactas

src/app/api/companies/tenant/route.ts + /mass-downloads/download-zip/route.ts @ tenant L69-L74, download-zip L35-L40

⚡ Nivel de Riesgo

Alto

🔍 Descripción del Riesgo

Ambos handlers retornan `NextResponse.json({ ..., details: error instanceof Error ? error.message : ... }, 500)`. Los mensajes de error de Prisma incluyen: nombres de tablas, nombres de columnas, restricciones UNIQUE violadas, paths de archivos .prisma. Los errores SAT incluyen `CodEstatus 5004` (bloqueo temporal), que son información valiosa de fingerprinting. Los errores de node incluyen paths locales (`C:\Users\rodri\...`) y nombres de librerías internas. Toda esta superficie reduce la complejidad de un ataque SQLi o blind.

🎯 Ejemplo de Exploit (PoC)

1. Provocar error de Prisma en `companies/tenant` enviando header `Accept` malformado o cookie corrupta (parseo de Prisma config).
2. `500: details: "Invalid `prisma.member.findMany()` invocation: ... table member not found"` (fingerprint tabla).
3. Usar ese leak para afinar UNION-based SQLi.

✅ Código Corregido

```
// [SAST-FIX API-07] Nunca exponga mensajes internos al cliente.
// Registrar TODO en console.error / pino / Datadog, pero response genérica.

try {
  // ... route logic ...
} catch (error) {
  console.error('[companies/tenant 500]', {
    message: error instanceof Error ? error.message : String(error),
    stack: error instanceof Error ? error.stack : undefined,
    userId: session?.user?.id || null
  })
}
```

```
    })  
    return NextResponse.json(  
      { error: 'Error interno del servidor. Si el problema persiste contacte a soporte.' },  
      { status: 500 }  
    )  
  }  
}
```

API-08

/mass-downloads/credentials FIEL Upload sin validación de contenido (MIME Spoofing + DoS)

Alto OWASP A02:2021 / A05:2021 · Cryptographic Failures + Security Misconfiguration

src/app/api/mass-downloads/credentials/route.ts L14-L23 (parseo formData) + L67-L87 (upsert credenciales)

📁 Archivo y Líneas exactas

src/app/api/mass-downloads/credentials/route.ts @ L14-L23 (parseo formData) + L67-L87 (upsert credenciales)

⚡ Nivel de Riesgo

Alto

🔍 Descripción del Riesgo

Al guardar credenciales FIEL (clave privada + cert + contraseña) el handler valida la existencia de fields L15-L23 y usa validateFiel para comprobar correspondencia, PERO: (a) No valida TAMAÑO MÁXIMO de privateKeyFile / certificateFile → un upload de 2 GB provoca OOM (Buffer.from L40-41). (b) ValidateFiel es bueno pero NO rechaza password vacíos o de 1 char antes de encriptar. (c) No guarda auditLog de quién actualizó credenciales → imposible forense si hay abuso. (d) No hay rate limit por organización → posibilidad de rotación de credenciales por fuerza bruta password intentos. (e) privateKey en Buffer se loguea si error contiene el buffer (no en este caso, pero detalles arriba), pero sí queda en memoria sin scrub.

🎯 Ejemplo de Exploit (PoC)

1. POST multipart/form-data privateKey = (2 GB de ceros binarios)
→ Node.js: "JavaScript heap out of memory" → servidor Next.js aborta. (DoS permanente sin rate limit).
2. Intentar password de 1 char mil veces por hora sin límite → validateFiel consume CPU sin freno.

✅ Código Corregido

```
export async function POST(req: NextRequest) {
  try {
    const session = await auth()
    if (!session?.user?.id) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })

    const formData = await req.formData()
    const rfc = formData.get("rfc") as string
```

```
const password = formData.get("password") as string
const privateKeyFile = formData.get("privateKey") as File
const certificateFile = formData.get("certificate") as File
const organizationId = formData.get("organizationId") as string

// [SAST-FIX API-08] Rate limit / organización
rateLimit({ key: `fiel-upload-${organizationId}`, interval: 60_000, limit: 10 })

if (!rfc || !password || !privateKeyFile || !certificateFile || !organizationId)
  return NextResponse.json({ error: 'Missing fields' }, { status: 400 })

// [SAST-FIX API-08] Validaciones estrictas de file size + password length
if (privateKeyFile.size > 32 * 1024) return NextResponse.json({ error: '.key > 32KB'
}, { status: 400 })
if (certificateFile.size > 128 * 1024) return NextResponse.json({ error: '.cer >
128KB' }, { status: 400 })
if (password.length < 8 || password.length > 128)
  return NextResponse.json({ error: 'Contraseña FIEL debe tener 8-128 caracteres' }, {
status: 400 })
```

API-09

/api/import sin validación de tamaño ni schema (XML Bomb / Billion Laughs)

Alto

OWASP A03:2021 / A08:2021 · Injection + Software & Data Integrity Failures

 src/app/api/import/route.ts L18-L48 (parseo body sin límites)

Archivo y Líneas exactas

src/app/api/import/route.ts @ L18-L48 (parseo body sin límites)

Nivel de Riesgo

Alto

Descripción del Riesgo

El handler acepta request.json() sin límite Next.js (default 1MB? pero notas L7-14 comentan que config fue deprecada; en App Router el default body limit es 1MB pero un atacante envía 100k requests paralelos). El XML que llega L40 se pasa a createInvoiceFromXml que probablemente usa libxmljs o fast-xml-parser: si no está protegido, un XML con ENTITY expandible ("Billion Laughs") agota la RAM en segundos. Además, array.length no tiene upper bound: enviar 10,000 items por lote causa 10,000 iteraciones síncronas.

Ejemplo de Exploit (PoC)

```
POST /api/import body = [{"xml": "<?xml version=\"1.0\"?><!DOCTYPE lolz [<!ENTITY lol1\n\"lol1\"><!ENTITY lol2\n\"&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;\n\">... ENTITY\nlol19\n\"&lol18;&lol18;&lol18;&lol18;&lol18;&lol18;&lol18;&lol18;&lol18;&lol18;\n\">]><lolz>&lol19;\n</lolz>"}]
```

→ Causa 10⁹ strings en memoria → Node.js aborta.

Código Corregido

```
// Usar zod para limitar lote + tamaño string de xml. Ver fixedCode de API-01 (schema z.array.max(500), xml.max(50MB)).
// Además, en createInvoiceFromXml, habilitar strict: true, processEntities: false en fast-xml-parser:
const xmlParserOpts = {
  ignoreAttributes: false,
  attributeNamePrefix: '@_',
  processEntities: false, // <== BLOQUEA Billion Laughs
  htmlEntities: false,
  allowBooleanAttributes: true,
  parseNodeValue: true,
```

```
trimValues: true  
}
```

API-10

Falta Rate Limit en endpoints de administración de queues y workers

Medio

OWASP A05:2021 · Security Misconfiguration

src/app/api/admin/sat-69b/sync/route.ts, src/app/api/tenant/update-progress/route.ts sat-69b L36-L47, update-progress L45-L46

Archivo y Líneas exactas

src/app/api/admin/sat-69b/sync/route.ts, src/app/api/tenant/update-progress/route.ts @ sat-69b L36-L47, update-progress L45-L46

⚡ Nivel de Riesgo

Medio

🔍 Descripción del Riesgo

Ambos endpoints POST llaman `sat69BBlacklistQueue.add()` o `updateTenantProgress()` SIN rate limit por organización ni usuario. Un Owner legítimo (cuyas credenciales fueron phisheadas) puede enviar 10,000 POST `/tenant/update-progress` por minuto. Cada invocación probablemente ejecuta consultas pesadas contra CFDI (agrupaciones por fecha, lecturas de XML). El worker BullMQ tiene `concurrency 5`, por lo que 10,000 jobs encolan saturando Redis y CPU del worker (DoS interno de la cola, imposibilitando jobs legítimos de otros tenants).

🔥 Ejemplo de Exploit (PoC)

```
Script paralelo async Pool(16) enviando POST /admin/sat-69b/sync X 200,000 veces.  
Cola sat69bBlacklistQueue crece indefinidamente; workers no atienden otros jobs. Escalado  
de costo (Cloud Provider Redis $$$) + out-of-memory en process worker.
```

✅ Código Corregido

```
// sat-69b/sync:  
export async function POST() {  
  try {  
    const session = await auth()  
    if (!session?.user?.id) return NextResponse.json({ error: 'No autorizado' }, { status:  
401 })  
    // membership guard ...  
  
    // [SAST-FIX API-10] 1 job por organización cada 10 min (idempotency)  
    const lastRun = await prisma.auditLog.findFirst({  
      where: { action: 'SAT69B_MANUAL_ENQUEUE', userId: session.user.id, timestamp: { gte:  
new Date(Date.now() - 10 * 60 * 1000) } },
```

```
        select: { id: true }
      })
      if (lastRun) return NextResponse.json({ error: 'Sincronización ya en ejecutada recientemente' }, { status: 429 })

      rateLimit({ key: `sat69b-org-${membership.organizationId}`, interval: 15 * 60 * 1000, limit: 2 })
      // ... create job + auditLog ...
    } catch (e) { /* ... */ }
  }
}
```


API-11

Endpoints /api/dev/* sin guardia de entorno global (apply a todo el segmento)

Medio

OWASP A05:2021 · Security Misconfiguration

 src/app/api/dev/ (toda la carpeta) middleware.ts / proxy / rewrite

Archivo y Líneas exactas

src/app/api/dev/ (toda la carpeta) @ middleware.ts / proxy / rewrite

Nivel de Riesgo

Medio

Descripción del Riesgo

Actualmente cada endpoint /api/dev/X tiene que implementar su propia guardia NODE_ENV. Esto es propenso a olvidos cuando se agrega un dev/X nuevo. El fix correcto es middleware-level: bloquear todas las rutas /api/dev/* cuando NODE_ENV=production, sin depender de que el handler individual lo haga.

Ejemplo de Exploit (PoC)

Agregar una nueva ruta /api/dev/reset-smtp sin guardia y subir a producción. Atacante hace POST /api/dev/reset-smtp y borra configuración SMTP global sin credenciales.

Código Corregido

```
// src/middleware.ts (agregar regla nueva)
const BLOCKED_PATHS_PROD: Array<RegExp> = [
  /^\\api\\dev(\\.*)?$/ // Bloquea todo /api/dev/* en production
]

export async function middleware(request: NextRequest) {
  if (process.env.NODE_ENV === 'production') {
    const pathname = request.nextUrl.pathname
    for (const p of BLOCKED_PATHS_PROD) {
      if (p.test(pathname)) {
        return NextResponse.json({ error: 'Endpoint no disponible' }, { status: 404 })
      }
    }
  }
}
```

```
// ... resto del middleware (auth / proxy / etc) ...  
}
```

API-12

/api/import console.error stack FULL + errorMessage al cliente XML leak

Medio OWASP A09:2021 / A02:2021 · Logging Failures / Cryptographic Failures

 src/app/api/import/route.ts L75-L85 (catch block) Archivo y Líneas exactas

src/app/api/import/route.ts @ L75-L85 (catch block)

 Nivel de Riesgo

Medio

 Descripción del Riesgo

El catch block loguea L77 Stack trace completo L78 y error message L79. Estos stacks incluyen paths locales: C:\Users\rodri\AppData\Local\Temp\build\src\lib\invoice-import.ts:132:22, nombres de funciones, UUIDs de invoice, valores raw de nodos XML de terceros. El campo L83 "details: errorMessage" devuelve al cliente el mismo mensaje de error. Si createInvoiceFromXml devuelve XML parseado (ej: "error en UUID X del RFC emisor Y"), se leakearía PII fiscal de otros registros fallidos al cliente sin scoping.

 Ejemplo de Exploit (PoC)

Enviar XML malicioso con valores de fuzzing (RFCs no pertenecientes). La respuesta 500: details: "Expected rfc=AAM010101AAA pero se recibió RFC=ROBBADO_OTRA_ORG, línea 45 col 12" le confirma al atacante qué otros RFC existen en la tabla.

 Código Corregido

```
catch (error: unknown) {
  // [SAST-FIX API-12] Loggear CON userId + request id PERO nunca leakear al cliente.
  const reqId = crypto.randomUUID()
  console.error('[import-batch 500]', {
    reqId,
    userId: session?.user?.id || 'ANON',
    message: error instanceof Error ? error.message : String(error),
    stack: error instanceof Error ? error.stack : undefined
  })
  return NextResponse.json(
    { error: `Error interno del servidor. ID de soporte: ${reqId}`, },
    { status: 500 }
  )
}
```

```
)  
}
```